

# SAT-Verified Discrete Physics: A Minimal, Auditable SAT Workflow for Finite Discrete Geometry and Locality

Oscar Riveros

Polímata, Chile

[independent.academia.edu/oarr](https://independent.academia.edu/oarr) • [github.com/maxtuno](https://github.com/maxtuno)

2025-12-29

## Abstract

We present a *methods paper*: a small, auditable workflow that compiles *finite discrete physics constraints* into SAT instances with mechanically checkable witnesses. The goal is *not* a continuum unification claim, but a reproducible way to (i) verify satisfiability of discrete model constraints, or (ii) refute them (UNSAT) within explicit finite regimes.

Two families of fully checkable examples are provided:

1. **Combinatorial 2D gravity (equilateral)**: we encode the existence of a closed simplicial 2-complex via triangle-incidence variables, enforce vertex valence constraints, and validate the discrete Gauss–Bonnet curvature sum. We include both SAT (octahedral sphere) and UNSAT (attempted “flat sphere”) instances.
2. **Boolean locality dynamics (finite-speed influence)**: a discrete-time influence propagation model on graphs that yields an exact light-cone constraint (a Boolean analogue of Lieb–Robinson locality). We provide SAT/UNSAT reachability instances parameterized by graph size and time horizon.

We report clause/variable growth under a one-hot reduction and include baseline solve times using a reference Python CDCL prototype, while emphasizing that the primary deliverable is the *auditable encoding plus validator*, compatible with industrial SAT solvers.

**Keywords.** SAT; discrete geometry; Regge calculus; Gauss–Bonnet; Lieb–Robinson bounds; constraint compilation; reproducible verification; symmetry breaking; cardinality encodings; GCNF.

## 1 Scope and claims (epistemic status)

This paper claims only the following:

- A **finite-domain compilation workflow** from discrete model constraints to SAT/MaxSAT.
- A **minimal intermediate format (GCNF)** that is easy to audit.
- A **verification toolchain** that checks witnesses against the original constraints.
- **Toy instances** that are non-trivial in the sense that they include both SAT and UNSAT cases and at least one instance found by search rather than hard-coded.

We explicitly *do not* claim: (i) a derivation of continuum GR/QFT from these toys, (ii) observational cosmology consequences, or (iii) that the presented instances are computationally hard at research scale.

## 1.1 Context within a larger discrete-physics program

The workflow is compatible with broader discrete programs (e.g., simplicial gravity, causal triangulations, operational locality constraints, and constraint-first “exact arithmetic” pipelines). In particular, it can serve as a verification substrate for any framework that treats physical claims as *finite refutable constraint sets* rather than as continuum axioms. This paper remains strictly at the “methods + finite instances” layer.

## 2 Related work (positioning)

**Regge calculus** introduced piecewise-linear discretizations of GR on simplicial complexes [1]. Numerical and structural work on Regge calculus and discrete gravity provides background on discretization choices and gauge issues [6, 7].

**Triangulation-based quantum gravity** (including causal dynamical triangulations) is a mature field; see the Physics Reports review [5] for context.

**Lieb–Robinson bounds** establish an effective finite velocity of information propagation in local quantum lattice systems [2, 3]. Our “Boolean locality dynamics” is not a bound in operator norm, but a deliberately simplified finite-speed propagation constraint family.

On the SAT side, reductions from finite-domain CSP to SAT and the design of SAT encodings for global constraints (especially **cardinality constraints**) are well studied. We use a standard sequential-counter style encoding for exact- $k$  constraints [4]. For optional background on encodings and comparisons, see e.g. [8, 9].

Our contribution is not a new gravity model or a new locality theorem; it is the **auditable bridge**:

$$\textit{finite discrete constraints} \rightarrow \textit{SAT instance} \rightarrow \textit{witness} \rightarrow \textit{mechanical validation},$$

with examples selected to demonstrate both verification and refutation.

## 3 Method: from discrete constraints to SAT

### 3.1 GCNF (Generalized CNF over equalities)

We use a simple clause language over finite-domain variables:

- Variables:  $x_i \in \{0, 1, \dots, b_i - 1\}$ .
- Atomic literal:  $x_i = a$ .
- Clause: disjunction of literals  $(x_{i_1} = a_1) \vee \dots \vee (x_{i_m} = a_m)$ .
- Formula: conjunction of clauses.

A **GCNF file** is a line-oriented list of Python-literal lists:

Listing 1: GCNF clause line format.

```
[(var_id, value), (var_id, value), ...]
```

Each line is one clause. This representation is intentionally boring: it is easy to audit by inspection and easy to translate to standard CNF.

### 3.2 One-hot reduction to CNF

For each multi-valued variable  $x$  with domain size  $b$ , introduce Boolean indicators  $X_{x,0}, \dots, X_{x,b-1}$  with **exactly-one** constraints. Then:

- The literal  $(x = a)$  becomes  $X_{x,a}$ .
- A GCNF clause becomes a standard CNF clause over these indicators.

The tool `gcnf_to_cnf.py` produces a DIMACS CNF file and a map file to translate between witnesses in CNF space and original domain assignments.

### 3.3 Verification loop (witness checking)

Given a SAT solver witness, we verify it **against the original GCNF constraints**, not only against the reduced CNF. The validator (`validate_solution.py`) checks every clause in the GCNF instance under the decoded assignment. This separates “solver correctness” from “encoding correctness” and makes the pipeline auditable.

### 3.4 Symmetry breaking as discrete gauge-fixing

Discrete constraint models often have large automorphism groups (e.g., vertex relabelings of triangulations). Adding **symmetry-breaking constraints** can reduce search dramatically, but must be treated as a *gauge choice* (it prunes equivalent solutions, not physics). We use optional symmetry breaking in some instances and discuss the risk of over-constraining (§7).

## 4 Toy-GR-Complex: synthesizing an equilateral 2-manifold and checking Gauss–Bonnet

### 4.1 Combinatorial curvature

For an **equilateral** triangulation, each incident triangle contributes angle  $\pi/3$  at a vertex, hence

$$K(v) = 2\pi - \deg(v) \frac{\pi}{3}, \quad \kappa(v) := 6 - \deg(v),$$

and the discrete Gauss–Bonnet identity becomes

$$\sum_v \kappa(v) = 6\chi.$$

For a triangulated 2-sphere ( $\chi = 2$ ) this yields  $\sum_v \kappa(v) = 12$ .

### 4.2 Encoding the existence of a closed simplicial 2-complex

Let the vertex set be  $V = \{1, \dots, N\}$ . For every triple  $i < j < k$ , introduce a Boolean variable

$$\tau_{ijk} \in \{0, 1\}$$

indicating whether the triangle  $(i, j, k)$  is present.

**Edge manifold constraint (no boundary).** For each unordered edge  $\{i, j\}$ , consider the set  $S_{ij} = \{\tau_{ijk} : k \neq i, j\}$ . We enforce that the number of incident triangles is **0 or 2**, i.e. (i) “at most 2” (forbid any 3 simultaneously true), and (ii) “not exactly 1” (if one is true, at least one other is true).

**Vertex valence constraint.** Define the vertex degree as the number of incident triangles:

$$\deg(v) = \sum_{i < j, i, j \neq v} \tau_{vij}.$$

We encode  $\deg(v) = d_v$  using an exact- $k$  sequential-counter CNF encoding [4].

#### 4.3 SAT example: $N = 6$ , $\deg(v) = 4$ (octahedral sphere)

Constraint target:  $\deg(v) = 4$  for all vertices. Then

$$\sum_v \kappa(v) = \sum_v (6 - 4) = 12,$$

consistent with a 2-sphere.

A satisfying witness found by SAT corresponds to the octahedron (8 triangles):

- (1, 2, 3)
- (1, 2, 6)
- (1, 3, 5)
- (1, 5, 6)
- (2, 3, 4)
- (2, 4, 6)
- (3, 4, 5)
- (4, 5, 6)

#### 4.4 UNSAT example: $N = 6$ , $\deg(v) = 6$ (“flat sphere” attempt)

Imposing  $\deg(v) = 6$  for all vertices would force  $\sum_v \kappa(v) = 0$ , i.e.  $\chi = 0$  (torus-like curvature budget), which contradicts the same closed-manifold constraints on  $N = 6$  in this encoding. SAT returns **UNSAT**, providing a concrete refutation in the finite regime.

#### 4.5 SAT example found by search: $N = 7$ , degree pattern (5, 5, 4, 4, 4, 4, 4)

Here  $\sum_v \deg(v) = 30$ , hence  $F = \frac{1}{3} \sum_v \deg(v) = 10$ , and  $\sum_v \kappa(v) = 6N - \sum_v \deg(v) = 42 - 30 = 12$ . SAT finds a triangulated sphere with 10 triangles:

- (1, 3, 6)
- (1, 3, 7)
- (1, 4, 5)
- (1, 4, 7)

- (1, 5, 6)
- (2, 3, 6)
- (2, 3, 7)
- (2, 4, 5)
- (2, 4, 7)
- (2, 5, 6)

This example is not hard-coded: the solver searches a 35-triangle candidate space and finds a consistent 2-manifold satisfying the degree constraints.

## 5 Toy-LR-Dynamics: Boolean finite-speed influence on graphs

### 5.1 Discrete influence model

Let  $G = (V, E)$  be a graph. Define Boolean variables  $I(v, t) \in \{0, 1\}$  indicating whether site  $v$  is “influenced” by time step  $t$ . Fix a time horizon  $T$ .

For each vertex  $v$  and step  $t \rightarrow t + 1$ , we impose the exact local update:

$$I(v, t + 1) \iff \left( I(v, t) \vee \bigvee_{u \in N(v)} I(u, t) \right).$$

This enforces a strict **light cone**: influence propagates at speed 1 edge per step, so

$$I(v, T) = 1 \Rightarrow d(v, v_0) \leq T$$

for a single initially influenced site  $v_0$ . This should be read as a finite deterministic locality constraint family inspired by the qualitative content of Lieb–Robinson locality [2, 3].

### 5.2 SAT/UNSAT reachability instances on paths

On a path graph of length  $L$  with initial site 1 influenced at  $t = 0$ , we ask whether site  $L$  can be influenced by time  $T$ .

- If  $T < L - 1$ : SAT must return **UNSAT** (outside the light cone).
- If  $T = L - 1$ : SAT returns **SAT** (the boundary of the light cone).

These instances provide a clean “refutation harness” for locality constraints: the solver cannot “cheat” by setting arbitrary future values because we enforce the reverse implication in the biconditional.

## 6 Benchmarks: size growth and baseline solve times

### 6.1 Summary table (representative instances)

Table 1 reports: number of GCNF clauses (lines), CNF size after one-hot reduction (variables/-clauses), and baseline solve time using the included Python CDCL prototype.

Note: the prototype solver is not an optimized research solver; clause/variable counts are the stable metric. Any DIMACS-compatible CDCL solver can be substituted.

An additional (larger) triangulation instance illustrates the rapid blow-up for search problems:

| Instance                                    | Result | Solve_s | GCNF_clauses | CNF_vars | CNF_clauses |
|---------------------------------------------|--------|---------|--------------|----------|-------------|
| LR path $L=8, T=3$                          | UNSAT  | 0.134   | 99           | 64       | 163         |
| LR path $L=8, T=7$                          | SAT    | 0.231   | 219          | 128      | 347         |
| Triangulation $N=6$ , deg=4                 | SAT    | 0.240   | 1441         | 1360     | 2801        |
| Triangulation $N=6$ , deg=6                 | UNSAT  | 0.270   | 1441         | 1360     | 2801        |
| Triangulation $N=7$ , (5, 5, 4, 4, 4, 4, 4) | SAT    | 4.377   | 3675         | 3430     | 7105        |

Table 1: Representative instances: sizes and baseline solve times.

| Instance                         | Result  | Solve_s | GCNF_clauses | CNF_vars | CNF_clauses |
|----------------------------------|---------|---------|--------------|----------|-------------|
| Triangulation $N=8$ , (5×4, 4×4) | TIMEOUT | 50      | 8120         | 7504     | 15624       |

Table 2: A larger triangulation instance: timeout on the reference prototype (time cap 50s).

## 6.2 Locality scaling on paths (family benchmark)

| $L$ | $T$ | Result | Solve_s | GCNF_clauses | CNF_vars | CNF_clauses |
|-----|-----|--------|---------|--------------|----------|-------------|
| 12  | 3   | UNSAT  | 0.128   | 151          | 96       | 247         |
| 16  | 4   | UNSAT  | 0.201   | 265          | 160      | 425         |
| 20  | 5   | UNSAT  | 0.265   | 411          | 240      | 651         |
| 24  | 6   | UNSAT  | 0.187   | 589          | 336      | 925         |

Table 3: Family benchmark: UNSAT reachability on paths.

**UNSAT reachability (outside the cone).**

**SAT reachability (on the cone boundary).**

## 6.3 A simple “journal-readiness” metric for instances

To make non-triviality and scalability assessable, we recommend reporting

$$\mathcal{M} = (V_{\max}, b_{\max}, C_{\max}, T_{\text{solve}}, \epsilon),$$

where  $V_{\max}$  is the largest vertex/graph size solved under a fixed time budget,  $b_{\max}$  is the largest palette/domain size used in metric encodings,  $C_{\max}$  is the largest CNF clause count handled,  $T_{\text{solve}}$  is a representative solve-time statistic, and  $\epsilon$  is a physics-meaningful approximation error (e.g. curvature error under palette discretization or constraint slack under MaxSAT). This paper reports  $V_{\max}$ ,  $C_{\max}$ , and  $T_{\text{solve}}$  for the toy families with Boolean domains ( $b = 2$ ); reporting  $\epsilon$  becomes meaningful once metric-palette encodings (non-Boolean  $b > 2$ ) are added.

# 7 Gauge, redundancy, and what “diffeomorphism” means here

## 7.1 Symmetry (label) gauge in combinatorial encodings

The triangulation synthesis model is invariant under vertex relabeling. This is a **pure gauge** in the encoding. Partial symmetry breaking (pinning a triangle, fixing a degree-5 vertex identity, etc.)

| $L$ | $T$ | Result | Solve_s | GCNF_clauses | CNF_vars | CNF_clauses |
|-----|-----|--------|---------|--------------|----------|-------------|
| 12  | 11  | SAT    | 0.155   | 519          | 288      | 807         |
| 16  | 15  | SAT    | 0.161   | 947          | 512      | 1459        |
| 20  | 19  | SAT    | 0.173   | 1503         | 800      | 2303        |
| 24  | 23  | SAT    | 0.249   | 2187         | 1152     | 3339        |

Table 4: Family benchmark: SAT reachability on the light-cone boundary.

can speed up SAT but may also **over-constrain** the instance if applied inconsistently (observed in practice for  $N = 7$  degree-pattern instances).

## 7.2 Regge gauge vs. what we encode

In full Regge calculus, edge lengths carry redundancies analogous to diffeomorphism freedom; small vertex displacements can manifest as approximate zero modes. In this paper’s **equilateral** toy, we intentionally collapse the metric to combinatorics, so the “gauge” is mostly **combinatorial automorphism**, not continuum diffeomorphism.

A next step toward Regge-length encodings (still finite) would use a discrete length palette and triangle-type lookup tables; gauge fixing could be implemented by pinning a subset of edge lengths or adding constraints that fix representative configurations. We treat this as future work (§8).

## 7.3 Continuum connection (what would be required)

To connect SAT-verified discrete invariants to continuum expectations one would need, at minimum:

- refinement sequences (increasing  $N$  or layered meshes),
- increasing metric palettes (approaching continuous lengths),
- and stable observables (curvature distributions, correlators) with convergence tests.

Nothing in this paper substitutes for that program; we only provide an auditable finite foundation on which such a program can be built.

# 8 Limitations and forward extensions

- **Hardness:** the instance families here are modest. Larger triangulation synthesis quickly becomes challenging without strong encodings (cardinality networks, PB constraints, symmetry handling) and industrial solvers.
- **MaxSAT / tolerances:** physical constraints often require bounded error; MaxSAT or pseudo-Boolean optimization is a natural extension.
- **Hamiltonian quantum dynamics:** the Boolean locality model captures causal support growth, not amplitudes or commutator norms. Bridging to full LR inequalities would require discretized norms and arithmetic constraints.
- **3D/4D:** extending the manifold-synthesis encoding to tetrahedra/4-simplices is straightforward in principle but quickly requires better encodings and symmetry control.

## 9 Software and data availability

The repository contains:

- `gcnf_to_cnf.py`: one-hot reduction to DIMACS + mapping,
- `validate_solution.py`: witness validation against original GCNF,
- `convert_solution.py`: mapping between CNF and domain assignments,
- `hess_cdcl.py`: a reference (prototype) CDCL-style solver for GCNF.

All toy instances in this paper can be generated deterministically by the scripts described in Appendix B, and validated by `validate_solution.py`.

## A Toolchain (minimal commands)

```
# 1) Convert GCNF -> DIMACS CNF (+ map)
python gcnf_to_cnf.py instance.gcnf instance.cnf --map instance.map

# 2) Solve CNF with your preferred SAT solver (example placeholder)
# cadical instance.cnf > sat.out

# 3) Convert CNF witness back to domain assignment (if needed)
python convert_solution.py --map instance.map --in sat.out --out instance.sol

# 4) Validate assignment against original GCNF constraints
python validate_solution.py --gcnf instance.gcnf --sol instance.sol
```

## B Instance generators (pseudocode)

### B.1 Triangulation synthesis (Toy-GR-Complex)

```
# Variables:  $\tau_{ijk}$  for each triple  $(i < j < k)$ 
# Constraints:
# (1) For each edge  $\{i, j\}$ : number of incident triangles in  $S_{ij}$  is 0 or 2
# (2) For each vertex  $v$ : sum of  $\tau_{vij}$  equals target degree  $d_v$  (cardinality encoding)
# Optional: symmetry breaking constraints (pin a triangle), used carefully.
```

### B.2 Boolean locality dynamics (Toy-LR-Dynamics)

```
# Variables:  $I(v, t)$  for  $v$  in  $V$ ,  $t=0..T$ 
# Base:  $I(v_0, 0)=1$  and  $I(v \neq v_0, 0)=0$ 
# Update:  $I(v, t+1) \leftrightarrow (I(v, t) \text{ OR } \bigvee_{u \in N(v)} I(u, t))$ 
# Query:  $I(\text{target}, T)=1$ 
```

## C GCNF clause example

A typical clause is a list of equality literals:

|                               |
|-------------------------------|
| $[(17, 0), (23, 1), (55, 1)]$ |
|-------------------------------|

interpreted as  $(x_{17} = 0) \vee (x_{23} = 1) \vee (x_{55} = 1)$ .

## References

- [1] T. Regge, General relativity without coordinates, *Il Nuovo Cimento* **19** (1961) 558–571. doi:[10.1007/BF02733251](https://doi.org/10.1007/BF02733251).
- [2] E. H. Lieb and D. W. Robinson, The finite group velocity of quantum spin systems, *Commun. Math. Phys.* **28** (1972) 251–257. doi:[10.1007/BF01645779](https://doi.org/10.1007/BF01645779).
- [3] B. Nachtergaele and R. Sims, Lieb–Robinson bounds and the exponential clustering theorem, *Commun. Math. Phys.* **265** (2006) 119–130. doi:[10.1007/s00220-006-1556-1](https://doi.org/10.1007/s00220-006-1556-1).
- [4] C. Sinz, Towards an optimal CNF encoding of Boolean cardinality constraints, in *CP 2005* (2005).
- [5] J. Ambjørn, A. Görlich, J. Jurkiewicz, and R. Loll, Nonperturbative quantum gravity, *Physics Reports* **519** (2012) 127–210. doi:[10.1016/S0370-1573\(12\)00301-8](https://doi.org/10.1016/S0370-1573(12)00301-8).
- [6] A. P. Gentle, Regge calculus: a unique tool for numerical relativity, *General Relativity and Gravitation* **34** (2002) 1701–1718.
- [7] S. H. Christiansen, On the linearization of Regge calculus, *Numerische Mathematik* **119** (2011) 613–640. doi:[10.1007/s00211-011-0394-z](https://doi.org/10.1007/s00211-011-0394-z).
- [8] Y. Feldman *et al.*, A systematic comparison of SAT encodings of linear constraints, in *CP 2024* (2024). (optional background)
- [9] M. Seidl *et al.*, pblib: A library for encoding pseudo-Boolean constraints into CNF, *SAT 2012 / later distributions*. (optional background)